

HerAI Fellowship

AI DEPLOYMENT

Modul Belajar Praktis dan Mudah Dipahami

Membawa model dari eksperimen menuju layanan produksi yang aman dan dapat dipantau.

Level	Intermediate
Estimasi belajar	18-22 jam
Prasyarat	Python, API dasar, container, dan machine learning dasar
Versi	1.0 - 23 Juli 2026
Route	/participant/courses/ai-deployment

Disusun untuk Participant Module - Project HerAI

Tentang Modul

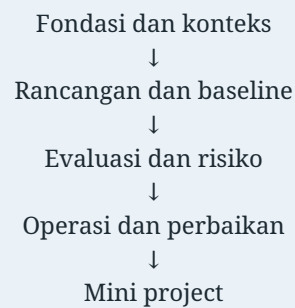
AI Deployment membahas packaging, serving, API contract, CI/CD, strategi rilis, monitoring, rollback, keamanan, dan incident response. Tujuan utamanya adalah membuat sistem AI tetap berguna setelah model pertama kali dipasang.

Pola belajar: pahami tujuan, buat baseline kecil, uji kegagalan, dokumentasikan bukti, lalu perbaiki secara bertahap. Jangan menambah kompleksitas sebelum masalah dan metriknya jelas.

Cara Menggunakan Modul

- Baca tujuan dan gambaran sederhana setiap bab.
- Kerjakan checkpoint tanpa melihat kembali materi.
- Adaptasi contoh pada data aman atau sintetis.
- Catat asumsi, error, keputusan, dan keterbatasan.
- Gunakan mini project sebagai artefak portofolio.

Peta Belajar



Daftar Isi

- 01 Bab 1 - Siklus Hidup Deployment AI**
- 02 Bab 2 - Packaging Model dan Registry
- 03 Bab 3 - Pola Batch, API, Stream, dan Edge
- 04 Bab 4 - Model Serving dan Resource Management
- 05 Bab 5 - API Contract dan Penanganan Error
- 06 Bab 6 - CI/CD dan Quality Gate
- 07 Bab 7 - Strategi Rilis dan Rollback
- 08 Bab 8 - Monitoring Model dan Data
- 09 Bab 9 - Optimasi Performa dan Biaya
- 10 Bab 10 - Keamanan dan Incident Response
- 11 Bab 11 - Mini Project: API Prediksi dengan Canary Release**
- 12 Bab 12 - Kuis Akhir**
- 13 Bab 13 - Diskusi dan Refleksi
- 14 Bab 14 - Glosarium dan Checklist
- 15 Bab 15 - Ringkasan dan Referensi**

HerAI Fellowship - Participant Module Membawa model dari eksperimen menuju layanan produksi yang aman dan dapat dipantau.

Tentang Modul

AI Deployment membahas packaging, serving, API contract, CI/CD, strategi rilis, monitoring, rollback, keamanan, dan incident response. Tujuan utamanya adalah membuat sistem AI tetap berguna setelah model pertama kali dipasang.

Modul disusun dengan bahasa bertahap. Setiap bab berisi tujuan, konsep inti, alur kerja, contoh kasus, kesalahan umum, checkpoint, dan latihan. Peserta tidak perlu menghafal seluruh istilah. Yang lebih penting adalah memahami hubungan antarkomponen dan mampu menjelaskan alasan di balik sebuah pilihan.

Capaian Pembelajaran

Setelah menyelesaikan modul, peserta mampu:

- menjelaskan fondasi dan istilah utama AI Deployment;
- menerjemahkan kebutuhan menjadi rancangan yang dapat diuji;
- membuat prototipe kecil dengan dokumentasi dan kontrol dasar;
- mengevaluasi kualitas, risiko, keterbatasan, serta biaya;
- menyampaikan hasil dan rekomendasi kepada pemangku kepentingan.

Prasyarat

- Python, API dasar, container, dan machine learning dasar.
- Mampu membaca diagram sederhana dan mengikuti contoh kode.
- Bersedia mencatat asumsi, kegagalan, dan keputusan selama latihan.

Cara Belajar yang Disarankan

1. Baca gambaran dan tujuan setiap bab.
2. Buat catatan istilah dengan bahasa sendiri.
3. Kerjakan checkpoint sebelum melihat kembali materi.
4. Jalankan atau adaptasi contoh kode bila relevan.
5. Gunakan mini project sebagai portofolio, bukan sekadar tugas akhir.
6. Lakukan review keamanan, privasi, dan dampak sebelum demonstrasi.

Peta Materi

Tahap	Fokus	Hasil
Fondasi	Istilah, tujuan, konteks, dan arsitektur	Peta masalah yang jelas
Pembangunan	Data, proses, komponen, dan integrasi	Baseline yang dapat diuji
Evaluasi	Kualitas, risiko, subgroup, biaya	Bukti dan daftar keterbatasan
Operasi	Monitoring, ownership, respons, iterasi	Sistem yang dapat dipelihara

Bab 1 - Siklus Hidup Deployment AI

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran siklus hidup deployment ai dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Siklus Hidup Deployment AI tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
artifact	konsep penting dalam siklus hidup deployment ai yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana artifact diukur, diuji, atau dikendalikan?
environment	konsep penting dalam siklus hidup deployment ai yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana environment diukur, diuji, atau dikendalikan?
release	konsep penting dalam siklus hidup deployment ai yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana release diukur, diuji, atau dikendalikan?
runtime	konsep penting dalam siklus hidup deployment ai yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana runtime diukur, diuji, atau dikendalikan?
feedback	konsep penting dalam siklus hidup deployment ai yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana feedback diukur, diuji, atau dikendalikan?
ownership	konsep penting dalam siklus hidup deployment ai yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana ownership diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **artifact** -> **environment** -> **release** -> **runtime**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **artifact** dapat memengaruhi **environment**, lalu mengubah cara **release** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan siklus hidup deployment ai dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk artifact serta environment.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan siklus hidup deployment ai secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **artifact** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan siklus hidup deployment ai.
- Menganggap artifact sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan artifact dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara environment dan release dalam bab ini?
3. Sebutkan satu risiko ketika siklus hidup deployment ai diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan artifact, environment, release, runtime. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 2 - Packaging Model dan Registry

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran packaging model dan registry dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Packaging Model dan Registry tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
serialization	konsep penting dalam packaging model dan registry yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana serialization diukur, diuji, atau dikendalikan?
dependency	konsep penting dalam packaging model dan registry yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana dependency diukur, diuji, atau dikendalikan?
image	konsep penting dalam packaging model dan registry yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana image diukur, diuji, atau dikendalikan?
registry	konsep penting dalam packaging model dan registry yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana registry diukur, diuji, atau dikendalikan?
provenance	informasi asal, versi, pemilik, dan riwayat sebuah artefak	Pertanyaan yang perlu dijawab: bagaimana provenance diukur, diuji, atau dikendalikan?
version	konsep penting dalam packaging model dan registry yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana version diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **serialization** -> **dependency** -> **image** -> **registry**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **serialization** dapat memengaruhi **dependency**, lalu mengubah cara **image** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan packaging model dan registry dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk serialization serta dependency.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan packaging model dan registry secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **serialization** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan packaging model dan registry.
- Menganggap serialization sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan serialization dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara dependency dan image dalam bab ini?
3. Sebutkan satu risiko ketika packaging model dan registry diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan serialization, dependency, image, registry. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 3 - Pola Batch, API, Stream, dan Edge

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran pola batch, api, stream, dan edge dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Pola Batch, API, Stream, dan Edge tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
batch	konsep penting dalam pola batch, api, stream, dan edge yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana batch diukur, diuji, atau dikendalikan?
online	konsep penting dalam pola batch, api, stream, dan edge yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana online diukur, diuji, atau dikendalikan?
stream	konsep penting dalam pola batch, api, stream, dan edge yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana stream diukur, diuji, atau dikendalikan?
edge	konsep penting dalam pola batch, api, stream, dan edge yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana edge diukur, diuji, atau dikendalikan?
latency	waktu yang dibutuhkan sistem untuk menghasilkan respons	Pertanyaan yang perlu dijawab: bagaimana latency diukur, diuji, atau dikendalikan?
availability	konsep penting dalam pola batch, api, stream, dan edge yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana availability diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **batch** -> **online** -> **stream** -> **edge**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **batch** dapat memengaruhi **online**, lalu mengubah cara **stream** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan pola batch, api, stream, dan edge dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk batch serta online.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan pola batch, api, stream, dan edge secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **batch** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan pola batch, api, stream, dan edge.
- Menganggap batch sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan batch dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara online dan stream dalam bab ini?
3. Sebutkan satu risiko ketika pola batch, api, stream, dan edge diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan batch, online, stream, edge. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 4 - Model Serving dan Resource Management

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran model serving dan resource management dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Model Serving dan Resource Management tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
server	konsep penting dalam model serving dan resource management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana server diukur, diuji, atau dikendalikan?
worker	konsep penting dalam model serving dan resource management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana worker diukur, diuji, atau dikendalikan?
concurrency	konsep penting dalam model serving dan resource management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana concurrency diukur, diuji, atau dikendalikan?
GPU	konsep penting dalam model serving dan resource management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana GPU diukur, diuji, atau dikendalikan?
autoscaling	konsep penting dalam model serving dan resource management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana autoscaling diukur, diuji, atau dikendalikan?
queue	konsep penting dalam model serving dan resource management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana queue diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **server** -> **worker** -> **concurrency** -> **GPU**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **server** dapat memengaruhi **worker**, lalu mengubah cara **concurrency** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan model serving dan resource management dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk server serta worker.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan model serving dan resource management secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **server** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan model serving dan resource management.
- Menganggap server sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan server dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara worker dan concurrency dalam bab ini?
3. Sebutkan satu risiko ketika model serving dan resource management diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan server, worker, concurrency, GPU. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 5 - API Contract dan Penanganan Error

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran api contract dan penanganan error dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

API Contract dan Penanganan Error tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
schema	aturan bentuk, tipe, dan makna field data	Pertanyaan yang perlu dijawab: bagaimana schema diukur, diuji, atau dikendalikan?
validation	konsep penting dalam api contract dan penanganan error yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana validation diukur, diuji, atau dikendalikan?
status code	konsep penting dalam api contract dan penanganan error yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana status code diukur, diuji, atau dikendalikan?
timeout	konsep penting dalam api contract dan penanganan error yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana timeout diukur, diuji, atau dikendalikan?
idempotency	sifat proses yang aman diulang tanpa menggandakan efek	Pertanyaan yang perlu dijawab: bagaimana idempotency diukur, diuji, atau dikendalikan?
compatibility	konsep penting dalam api contract dan penanganan error yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana compatibility diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **schema** -> **validation** -> **status code** -> **timeout**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **schema** dapat memengaruhi **validation**, lalu mengubah cara **status code** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan api contract dan penanganan error dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk schema serta validation.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan api contract dan penanganan error secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **schema** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan api contract dan penanganan error.
- Menganggap schema sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan schema dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara validation dan status code dalam bab ini?
3. Sebutkan satu risiko ketika api contract dan penanganan error diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan schema, validation, status code, timeout. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 6 - CI/CD dan Quality Gate

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran ci/cd dan quality gate dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

CI/CD dan Quality Gate tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
build	konsep penting dalam ci/cd dan quality gate yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana build diukur, diuji, atau dikendalikan?
test	konsep penting dalam ci/cd dan quality gate yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana test diukur, diuji, atau dikendalikan?
scan	konsep penting dalam ci/cd dan quality gate yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana scan diukur, diuji, atau dikendalikan?
deploy	konsep penting dalam ci/cd dan quality gate yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana deploy diukur, diuji, atau dikendalikan?
approval	konsep penting dalam ci/cd dan quality gate yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana approval diukur, diuji, atau dikendalikan?
artifact promotion	konsep penting dalam ci/cd dan quality gate yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana artifact promotion diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **build** -> **test** -> **scan** -> **deploy**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **build** dapat memengaruhi **test**, lalu mengubah cara **scan** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan ci/cd dan quality gate dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk build serta test.
4. Bangun versi kecil menggunakan data atau skenario yang aman.

5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan ci/cd dan quality gate secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **build** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan ci/cd dan quality gate.
- Menganggap build sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan build dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara test dan scan dalam bab ini?
3. Sebutkan satu risiko ketika ci/cd dan quality gate diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan build, test, scan, deploy. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 7 - Strategi Rilis dan Rollback

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran strategi rilis dan rollback dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Strategi Rilis dan Rollback tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
blue-green	konsep penting dalam strategi rilis dan rollback yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana blue-green diukur, diuji, atau dikendalikan?
canary	rilis bertahap ke sebagian kecil trafik sebelum diperluas	Pertanyaan yang perlu dijawab: bagaimana canary diukur, diuji, atau dikendalikan?
shadow	konsep penting dalam strategi rilis dan rollback yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana shadow diukur, diuji, atau dikendalikan?
feature flag	konsep penting dalam strategi rilis dan rollback yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana feature flag diukur, diuji, atau dikendalikan?
rollback	mengembalikan sistem ke versi stabil sebelumnya	Pertanyaan yang perlu dijawab: bagaimana rollback diukur, diuji, atau dikendalikan?
migration	konsep penting dalam strategi rilis dan rollback yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana migration diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **blue-green** -> **canary** -> **shadow** -> **feature flag**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **blue-green** dapat memengaruhi **canary**, lalu mengubah cara **shadow** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan strategi rilis dan rollback dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk blue-green serta canary.
4. Bangun versi kecil menggunakan data atau skenario yang aman.

5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan strategi rilis dan rollback secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **blue-green** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan strategi rilis dan rollback.
- Menganggap blue-green sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan blue-green dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara canary dan shadow dalam bab ini?
3. Sebutkan satu risiko ketika strategi rilis dan rollback diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan blue-green, canary, shadow, feature flag. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 8 - Monitoring Model dan Data

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran monitoring model dan data dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Monitoring Model dan Data tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
latency	waktu yang dibutuhkan sistem untuk menghasilkan respons	Pertanyaan yang perlu dijawab: bagaimana latency diukur, diuji, atau dikendalikan?
error	konsep penting dalam monitoring model dan data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana error diukur, diuji, atau dikendalikan?
drift	perubahan distribusi data atau hubungan yang dipelajari model	Pertanyaan yang perlu dijawab: bagaimana drift diukur, diuji, atau dikendalikan?
quality	konsep penting dalam monitoring model dan data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana quality diukur, diuji, atau dikendalikan?
feedback	konsep penting dalam monitoring model dan data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana feedback diukur, diuji, atau dikendalikan?
alert	konsep penting dalam monitoring model dan data yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana alert diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **latency** -> **error** -> **drift** -> **quality**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **latency** dapat memengaruhi **error**, lalu mengubah cara **drift** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan monitoring model dan data dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk latency serta error.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan monitoring model dan data secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **latency** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan monitoring model dan data.
- Menganggap latency sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan latency dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara error dan drift dalam bab ini?
3. Sebutkan satu risiko ketika monitoring model dan data diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan latency, error, drift, quality. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 9 - Optimasi Performa dan Biaya

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran optimasi performa dan biaya dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Optimasi Performa dan Biaya tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
batching	konsep penting dalam optimasi performa dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana batching diukur, diuji, atau dikendalikan?
caching	konsep penting dalam optimasi performa dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana caching diukur, diuji, atau dikendalikan?
quantization	konsep penting dalam optimasi performa dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana quantization diukur, diuji, atau dikendalikan?
throughput	jumlah pekerjaan yang dapat diselesaikan per satuan waktu	Pertanyaan yang perlu dijawab: bagaimana throughput diukur, diuji, atau dikendalikan?
utilization	konsep penting dalam optimasi performa dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana utilization diukur, diuji, atau dikendalikan?
cost	konsep penting dalam optimasi performa dan biaya yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana cost diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **batching** -> **caching** -> **quantization** -> **throughput**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **batching** dapat memengaruhi **caching**, lalu mengubah cara **quantization** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan optimasi performa dan biaya dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk batching serta caching.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan optimasi performa dan biaya secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **batching** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan optimasi performa dan biaya.
- Menganggap batching sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan batching dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara caching dan quantization dalam bab ini?
3. Sebutkan satu risiko ketika optimasi performa dan biaya diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan batching, caching, quantization, throughput. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 10 - Keamanan dan Incident Response

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran keamanan dan incident response dalam AI Deployment;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Keamanan dan Incident Response tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Deployment seperti membuka restoran dari resep percobaan: selain rasa, dibutuhkan dapur, standar porsi, antrean, kontrol kualitas, keamanan, dan prosedur ketika terjadi masalah.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
authentication	konsep penting dalam keamanan dan incident response yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana authentication diukur, diuji, atau dikendalikan?
authorization	konsep penting dalam keamanan dan incident response yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana authorization diukur, diuji, atau dikendalikan?
rate limit	konsep penting dalam keamanan dan incident response yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana rate limit diukur, diuji, atau dikendalikan?
audit	konsep penting dalam keamanan dan incident response yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana audit diukur, diuji, atau dikendalikan?
incident	konsep penting dalam keamanan dan incident response yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana incident diukur, diuji, atau dikendalikan?
postmortem	konsep penting dalam keamanan dan incident response yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana postmortem diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **authentication** -> **authorization** -> **rate limit** -> **audit**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **authentication** dapat memengaruhi **authorization**, lalu mengubah cara **rate limit** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan keamanan dan incident response dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk authentication serta authorization.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **API Prediksi dengan Canary Release**, tim perlu menerapkan keamanan dan incident response secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **authentication** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan keamanan dan incident response.
- Menganggap authentication sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan authentication dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara authorization dan rate limit dalam bab ini?
3. Sebutkan satu risiko ketika keamanan dan incident response diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan authentication, authorization, rate limit, audit. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 11 - Mini Project: API Prediksi dengan Canary Release

Tujuan Proyek

Mengemas model sebagai API, menambahkan validasi, health check, monitoring, canary rollout, serta prosedur rollback.

Proyek harus cukup kecil untuk selesai, tetapi cukup lengkap untuk memperlihatkan alur berpikir. Nilai utama bukan pada tampilan atau kompleksitas, melainkan kejelasan tujuan, kualitas keputusan, pengujian, dan dokumentasi.

Deliverable

- problem statement dan intended use;
- diagram arsitektur atau alur kerja;
- data dictionary atau inventaris sumber;
- baseline yang dapat dijalankan;
- hasil pengujian normal, error, dan kasus batas;
- daftar risiko serta kontrol;
- ringkasan hasil untuk audiens nonteknis;
- README dan rencana pengembangan.

Tahapan Pengerjaan

1. Tulis masalah, pengguna, keputusan, dan batas penggunaan.
2. Buat inventaris data, sumber daya, pemilik, izin, serta risiko.
3. Rancang arsitektur atau alur kerja versi minimum.
4. Bangun baseline yang dapat dijalankan ulang.
5. Tambahkan validasi, logging, dan penanganan error.
6. Susun golden set atau skenario uji, termasuk kasus batas.
7. Ukur outcome, guardrail, performa, dan biaya.
8. Dokumentasikan hasil, keterbatasan, runbook, serta rencana iterasi.

Contoh Kode Awal

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class Request(BaseModel):
    age: int = Field(ge=0, le=120)
    monthly_spend: float = Field(ge=0)

@app.get("/health")
def health():
    return {"status": "ok", "model_version": "1.0.0"}

@app.post("/predict")
def predict(payload: Request):
    score = model.predict_proba([[payload.age, payload.monthly_spend]])[0, 1]
    return {"score": float(score), "model_version": "1.0.0"}
```

Kode tersebut hanya titik awal. Tambahkan pemeriksaan input, konfigurasi, test, logging, serta penanganan error sesuai konteks proyek.

Rubrik Penilaian

Aspek	Bobot	Indikator
Problem framing	15%	Pengguna, keputusan, outcome, dan batas jelas
Data atau input	15%	Sumber, izin, kualitas, dan representasi dijelaskan
Rancangan	15%	Komponen, interface, serta trade-off

Aspek	Bobot	Indikator
		masuk akal
Implementasi	20%	Baseline berjalan, modular, dan dapat diulang
Evaluasi	15%	Metrik, kasus batas, serta subgroup diperiksa
Responsible practice	10%	Privasi, keamanan, fairness, dan human review
Komunikasi	10%	Dokumentasi dan demo mudah dipahami

Pertanyaan Review Proyek

1. Keputusan apa yang benar-benar dibantu proyek ini?
2. Apa kegagalan yang paling mungkin dan paling berbahaya?
3. Bukti apa yang menunjukkan solusi lebih baik daripada baseline?
4. Siapa yang berwenang menyetujui, menghentikan, atau mengubah sistem?
5. Informasi apa yang harus terlihat oleh pengguna agar tidak salah memahami hasil?

Bab 12 - Kuis Akhir

Pilih jawaban yang paling tepat.

1. Ketika memulai **Siklus Hidup Deployment AI**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji artifact
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
2. Ketika memulai **Packaging Model dan Registry**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji serialization
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
3. Ketika memulai **Pola Batch, API, Stream, dan Edge**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji batch
 - D. Mengukur keberhasilan hanya dari jumlah fitur
4. Ketika memulai **Model Serving dan Resource Management**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji server
5. Ketika memulai **API Contract dan Penanganan Error**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji schema
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
6. Ketika memulai **CI/CD dan Quality Gate**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji build
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
7. Ketika memulai **Strategi Rilis dan Rollback**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji blue-green
 - D. Mengukur keberhasilan hanya dari jumlah fitur
8. Ketika memulai **Monitoring Model dan Data**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji latency
9. Ketika memulai **Optimasi Performa dan Biaya**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji batching
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
10. Ketika memulai **Keamanan dan Incident Response**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji authentication
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur

11. Input penting berubah tanpa pemberitahuan. Kontrol terbaik adalah ...
- menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - Kontrak, validasi, versioning, dan alert
 - menghapus logging untuk menghemat ruang
12. Hasil sistem terlihat baik pada data latihan tetapi buruk pada konteks baru. Langkah terbaik adalah ...
- menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
 - Memeriksa split, leakage, representasi data, dan generalisasi
13. Tim tidak tahu siapa yang harus merespons alarm. Perbaikan utama adalah ...
- Menetapkan ownership, severity, runbook, dan escalation
 - menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
14. Perubahan besar akan diluncurkan. Cara menurunkan blast radius adalah ...
- menambah kompleksitas tanpa diagnosis
 - Rilis bertahap, observasi, dan rollback
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
15. Pengguna tidak memahami keterbatasan hasil. Desain yang tepat adalah ...
- menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - Menjelaskan sumber, ketidakpastian, batas penggunaan, dan koreksi
 - menghapus logging untuk menghemat ruang
16. Data sensitif digunakan untuk latihan. Prinsip pertama adalah ...
- menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
 - Minimization, izin, kontrol akses, dan penggunaan data sintetis bila memungkinkan
17. Satu metrik meningkat tetapi dampak keseluruhan memburuk. Tim perlu ...
- Menggunakan outcome metric dan guardrail secara bersamaan
 - menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
18. Sistem gagal pada sebagian kelompok. Respons yang benar adalah ...
- menambah kompleksitas tanpa diagnosis
 - Menganalisis subgroup, dampak, akar masalah, dan mitigasi
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
19. Biaya naik tanpa kenaikan nilai. Langkah awal adalah ...
- menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - Mengukur penggunaan per komponen dan menghubungkannya dengan outcome
 - menghapus logging untuk menghemat ruang
20. Temuan belum cukup kuat untuk keputusan otomatis. Pilihan aman adalah ...
- menambah kompleksitas tanpa diagnosis
 - menyembunyikan error dari pengguna
 - menghapus logging untuk menghemat ruang
 - Membatasi penggunaan sebagai dukungan keputusan dan menambahkan human review

Kunci dan Pembahasan

No.	Jawaban	Pembahasan
1	A	Pendekatan dimulai dari tujuan dan

No.	Jawaban	Pembahasan
		definisi operasional artifact, kemudian alat dipilih sesuai kebutuhan.
2	B	Pendekatan dimulai dari tujuan dan definisi operasional serialization, kemudian alat dipilih sesuai kebutuhan.
3	C	Pendekatan dimulai dari tujuan dan definisi operasional batch, kemudian alat dipilih sesuai kebutuhan.
4	D	Pendekatan dimulai dari tujuan dan definisi operasional server, kemudian alat dipilih sesuai kebutuhan.
5	A	Pendekatan dimulai dari tujuan dan definisi operasional schema, kemudian alat dipilih sesuai kebutuhan.
6	B	Pendekatan dimulai dari tujuan dan definisi operasional build, kemudian alat dipilih sesuai kebutuhan.
7	C	Pendekatan dimulai dari tujuan dan definisi operasional blue-green, kemudian alat dipilih sesuai kebutuhan.
8	D	Pendekatan dimulai dari tujuan dan definisi operasional latency, kemudian alat dipilih sesuai kebutuhan.
9	A	Pendekatan dimulai dari tujuan dan definisi operasional batching, kemudian alat dipilih sesuai kebutuhan.
10	B	Pendekatan dimulai dari tujuan dan definisi operasional authentication, kemudian alat dipilih sesuai kebutuhan.
11	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
12	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
13	A	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
14	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
15	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
16	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.

No.	Jawaban	Pembahasan
17	A	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
18	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
19	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
20	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.

Interpretasi Skor

- **17-20 benar:** siap mengerjakan proyek dengan pengawasan ringan.
- **13-16 benar:** pemahaman baik, ulangi bagian yang masih lemah.
- **9-12 benar:** pelajari kembali konsep inti dan latihan.
- **0-8 benar:** ulangi modul secara bertahap dengan mentor atau teman belajar.

Bab 13 - Diskusi dan Refleksi

Pertanyaan Diskusi

1. Bagian mana dari AI Deployment yang paling mudah diremehkan tetapi berpotensi menimbulkan kegagalan besar?
2. Kapan solusi sederhana lebih baik daripada solusi yang lebih otomatis atau kompleks?
3. Metrik apa yang dapat mendorong perilaku salah bila digunakan sendirian?
4. Bagaimana pengguna dapat mengoreksi sistem dan melihat tindak lanjutnya?
5. Siapa yang menerima manfaat dan siapa yang mungkin menerima risiko?

Jurnal Refleksi

Tuliskan satu halaman yang menjawab:

- konsep yang paling mengubah cara berpikir;
- asumsi yang sebelumnya dianggap benar;
- kesalahan yang terjadi ketika latihan;
- bukti yang masih kurang;
- satu tindakan belajar berikutnya.

Bab 14 - Glosarium dan Checklist

Glosarium

Istilah	Makna ringkas
artifact	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
environment	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
release	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
runtime	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
feedback	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
ownership	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
serialization	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
dependency	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
image	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
registry	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
provenance	informasi asal, versi, pemilik, dan riwayat sebuah artefak.
version	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
batch	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
online	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
stream	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
edge	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
latency	waktu yang dibutuhkan sistem untuk menghasilkan respons.
availability	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
server	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
worker	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
concurrency	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
GPU	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.

Istilah	Makna ringkas
autoscaling	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
queue	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
schema	aturan bentuk, tipe, dan makna field data.
validation	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
status code	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
timeout	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
idempotency	sifat proses yang aman diulang tanpa menggandakan efek.
compatibility	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
build	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
test	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
scan	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
deploy	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
approval	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
artifact promotion	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
blue-green	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
canary	rilis bertahap ke sebagian kecil trafik sebelum diperluas.
shadow	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
feature flag	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
rollback	mengembalikan sistem ke versi stabil sebelumnya.
migration	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
error	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.
drift	perubahan distribusi data atau hubungan yang dipelajari model.
quality	konsep penting dalam ai deployment yang perlu diberi definisi operasional sebelum dipakai.

Checklist Sebelum Implementasi

- [] Masalah, pengguna, dan keputusan telah dinyatakan.
- [] Intended use dan penggunaan yang dilarang telah ditulis.

- [] Sumber data atau input memiliki izin dan provenance.
- [] Baseline dan kriteria penerimaan tersedia.
- [] Kondisi error, data kosong, dan kasus ekstrem diuji.
- [] Privasi, keamanan, bias, dan aksesibilitas ditinjau.
- [] Log, metric, owner, alert, dan runbook ditetapkan.
- [] Perubahan memiliki versioning, approval, dan rollback.
- [] Hasil dikomunikasikan bersama keterbatasannya.
- [] Jadwal review dan penghentian sistem ditentukan.

Bab 15 - Ringkasan dan Referensi

Ringkasan Cepat

1. Mulai dari keputusan dan pengguna, bukan alat.
2. Definisikan istilah, data, outcome, dan guardrail secara operasional.
3. Bangun baseline kecil yang dapat diamati dan diuji ulang.
4. Perlakukan keamanan, privasi, aksesibilitas, serta human review sebagai bagian desain.
5. Uji kondisi normal, kegagalan, subgroup, perubahan konteks, dan biaya.
6. Dokumentasikan provenance, asumsi, versi, owner, serta keterbatasan.
7. Gunakan monitoring dan feedback untuk memutuskan iterasi, rollback, atau penghentian.

Referensi Utama

- Machine Learning Engineering - Andriy Burkov.
- Google Rules of Machine Learning.
- Kubernetes Documentation.
- OpenTelemetry Documentation.
- NIST Secure Software Development Framework.

Penutup

Kemampuan dalam AI Deployment tidak hanya ditunjukkan oleh banyaknya alat yang dikuasai. Kemampuan terlihat dari cara peserta merumuskan masalah, memilih pendekatan yang proporsional, menguji klaim, melindungi pengguna, dan menjaga sistem tetap dapat dipertanggungjawabkan.

HerAI Fellowship - Participant Module Versi 1.0.0 - 23 Juli 2026